8.5.2 Testing data persistence with @DataR2dbcTest and Testcontainers

As you might recall from the previous chapters, Spring Boot allows you to run integration tests by loading only the Spring components used by a specific application slice. For the REST API, you will create tests for the WebFlux slice. Here I'll show you how to write tests for the R2DBC slice using the `@DataR2dbcTest` annotation.

The approach is the same you used in chapter 5 for testing the data layer in Catalog Service, but there are two main differences. First, you will use the `StepVerifier` utility to test the `OrderRepository` behavior reactively. Second, you will define a PostgreSQL test container instance explicitly.

For the Catalog Service application, we relied on the test container auto-configuration. In this case, we'll define a test container in the test class and mark it as `@Container`. Then the `@Testcontainers` annotation on the class will activate the automatic startup and cleanup of the test container. Finally, we'll use the `@DynamicProperties` annotation provided by Spring Boot to pass the test database's credentials and URL to the application. This approach of defining test containers and overwriting properties is generic and can be applied to other scenarios.

Now, on to the code. Create an `OrderRepositoryR2dbcTests` class and implement autotests to verify the data persistence layer of the application.

Listing 8.30 Integration tests for the Data R2DBC slice

```
package com.polarbookshop.orderservice.order.domain;
```